DHOLE PATIL COLLEGE OF ENGINEERING
Wagholi, Pune
Department of E&TC Engineering



LABORATORY MANUAL

2024-2025

LAB PRACTICE-II

BE-E&TC ENGINEERING SEMESTER-II

Subject Code: **404187**

TEACHING SCHEME	EXAMINATION SCHEME
Lectures:- 4Hrs/Week	Practical:- 50 Marks
Practical:- 2 Hrs/Week	Term Work:- 25 Marks

Name of Faculty :-

Mrs. S. S. Khartad.
Department of E&TC Engineering
DPES College of Engineering

Work Load	Exam Schemes		
Practical	**Term Work**	**Practical**	**Oral**
02 hrs per week	**25**	**50**	**--**

List of Assignments

Sr. No.	Title of Experiment	Time Span (No. of weeks)
1	To write VHDL code for 4 bit ALU for add, subtract, AND, NAND, XOR, XNOR, OR, & ALU pass.	**02**
2	To write VHDL code for universal shift register with mode selection input for SISO, SIPO, PISO, & PIPO modes. Simulate with test bench, synthesis, implement on PLD.	**01**
3	To write VHDL code for FIFO memory. Simulate with test bench, synthesis, implement on PLD.	**01**
4	To write VHDL code for LCD interface. Simulate with test bench, synthesis, implement on PLD.	**02**
5	To write VHDL code for Up and Down Counter. Simulate with test bench,synthesis, implement on PLD	
6	To prepare CMOS layout for Inverter, NAND, NOR gates, Half Adder. Simulate with and without capacitive load, comment on rise, and fall times.	**02**
7	To prepare CMOS layout for 2:1 multiplexer using logic gates and transmission gates. Simulate with and without capacitive load, comment on rise, and fall times.	**01**
8	To prepare CMOS layout for Single bit SRAM cell. Simulate with and without capacitive load, comment on rise, and fall times.	**01**
9	To prepare CMOS layout for Half adder and Full Adder. Simulate with and withoutcapacitive load, comment on rise, and fall times.	**01**

Text Books:

1. Charles H. Roth, " Digital Systems design using VHDL", PWS
2. Wyane Wolf, "Modern VLSI Design (System on Chip)", PHI Publication.

Reference Books:

1. Allen Hollberg, "Analog CMOS Design", Oxford University Press.
2. Neil H.E.Weste, "David Money Harris, "CMOS VLSI Design: A Circuit & System Perspective", Pearson Publication.
3. Douglas Perry, "VHDL Programming By Example", 4edition, Tata McGraw Hill Publication.
4. Stephen Brown, "Fundamentals of Digital Logic with Verilog Design, Tata McGraw Hill Publication.
5. Behzad Razavi "Design of Analog CMOS Design Integrated circuit", Tata McGraw Hill Publication.

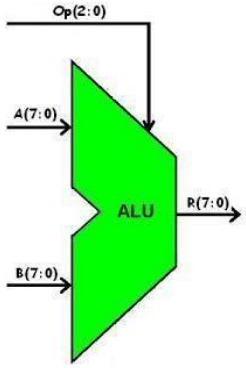
Experiment No.1

Title: To write VHDL code for 4 bit ALU for add, subtract, AND, NAND, XOR, XNOR, OR, & ALU pass.

PROBLEM STATEMENT:
To write VHDL code for 4 bit ALU for add, subtract, AND, NAND, XOR, XNOR, OR, & ALU pass.

OBJECTIVE:
To study the design flow of VHDL language

THEORY:
ALU(Arithmetic Logic Unit) is a digital circuit which does arithmetic and logical operations. Its a basic block in any processor. Note that this is one of the simplest architecture of an ALU. Most of the ALU's used in practical designs are far more complicated.
The block diagram of the ALU is given below. It receives two input operands 'A' and 'B' which are 8 bits long. The result is denoted by 'R' which is also 8 bit long. The input signal 'Op' is a 3 bit value which tells the ALU what operation to be performed by the ALU. Since 'Op' is 3 bits long we can have 2^3=8 operations.



ALU is capable of doing the following operations:

ALU Operation	Description
Add Signed	R = A + B: Treating A, B, and R as signed two's complement integers.
Subtract Signed	R = A - B: Treating A, B, and R as signed two's complement integers.
Bitwise AND	R(i) = A(i) AND B(i).
Bitwise NAND	R(i) = A(i) NAND B(i).
Bitwise OR	R(i) = A(i) OR B(i).
Bitwise XNOR	R(i) = A(i) XNOR B(i).
Bitwise XOR	R(i) = A(i) XOR B(i).
ALU PASS	R(i) = A(i).

These functions are implemented using a case statement. The ALU calculates the outputs at every positive edge of clock cycle. As soon as the outputs are calculated it is available at the portsignal 'R'.

PLATFORM USED:

1. Xilinx 9.1 ISE Design Suite
2. Spartan 3E XC3s250E 4C

FAQ:

What do you mean by HDLs?

Hardware description language or HDL is any language from a class of computer languages and/or programming languages for formal description of electronic circuits, and more specifically, digital logic. It can describe the circuit's operation, its design and organization, and tests to verify its operation by means of simulation. HDLs are standard text-based expressions of the spatial and temporal structure and behavior of electronic systems. Like concurrent programming languages, HDL syntax and semantics includes explicit notations for expressing concurrency. However, in contrast to most software programming languages, HDLs also include an explicit notion of time, which is a primary attribute of hardware Languages whose only characteristic is to express circuit connectivity between a hierarchy of blocks are properly classified as netlist languages used on electric computer-aided design .VHDL and VERILOG are the two most widely used Hardware description languages .

What is VLSI Design?

VLSI Design stands for Very Large scale Integrated circuit design. VLSI is the process of creating integrated circuits by combining thousands of transistor-based circuits into a single chipVLSI which involves the packing of more and more logic devices into smaller and smaller areas.

CONCLUSION:

Experiment No. 2

Title: To write VHDL code for universal shift register with mode selection input for SISO, SIPO, PISO, & PIPO modes. Simulate with test bench, synthesis, implement on PLD.

PROBLEM STATEMENT:
To write VHDL code for universal shift register with mode selection input for SISO, SIPO, PISO, & PIPO modes. Simulate with test bench, synthesis, implement on PLD.

OBJECTIVE:
To understand the concept of hardware description language.

THEORY:

SHIFT REGISTERS

Universal Shift Register
A universal shift register is an integrated logic circuit that can transfer data in four different modes. Like a parallel register it can load and transmit data in parallel. Like shift registers it can load and transmit data in serial fashions, through left shifts or right shifts. In addition, the universal shift register can combine the capabilities of both parallel and shift registers to accomplish tasks that neither basic type of register can perform on its own. For instance, on a particular job a universal register can load data in series (e.g. through a sequence of left shifts) and then transmit/output data in parallel.
Universal shift registers, as all other types of registers, are used in computers as memory elements. Although other types of memory devices are used for the efficient storage of very largevolume of data, from a digital system perspective when we say computer memory we mean registers. In fact, all the operations in a digital system are performed on registers. Examples of such operations include multiplication, division, and data transfer.
In order for the universal shift register to operate in a specific mode, it must first select the mode. To accomplish mode selection the universal register uses a set of two selector switches, S1 and S0. As shown in Table 1, each permutation of the switches corresponds to a loading/input mode.

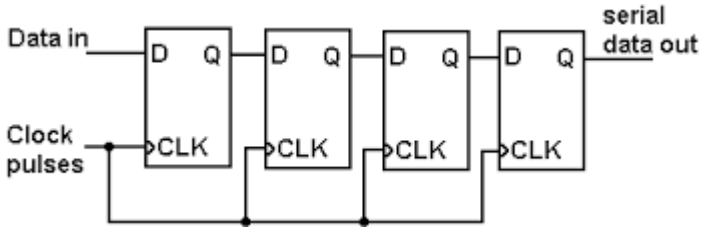
Operating Mode	S1	S0
SISO	0	0
SIPO	0	1
PISO	1	0
PIPO	1	1

Table 1

Serial In, Serial Out Shift Register

Since 4 flip flops are used, the register can store 4 bits of data. Serial data is applied at the D input of the first FF. The Q output of the first FF is connected to the D input of the second FF and so on. The output data is taken from the Q terminal of the last flip flop.

When serial data is transferred into a register, each new bit is clocked into the first FF at the positive going edge of each clock pulse. The bit that was previously stored by the first FF is transferred to the second FF. The bit that was stored by the second FF is transferred to the third FF and so on. The bit that was stored by the last FF is shifted out.



For N bit of data,

Number of clock pulse required = 2N – 1

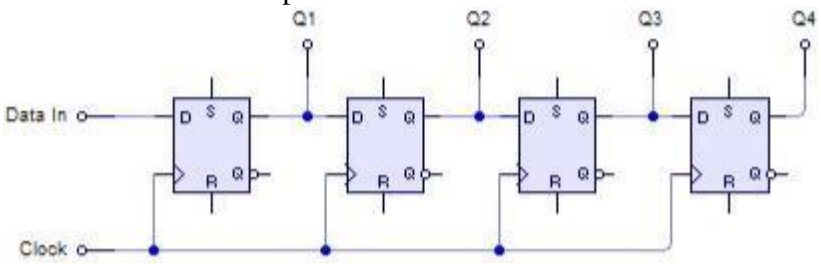
Total time required = (2N - 1) T

Serial in, Parallel out Shift Register

This configuration allows conversion from serial to parallel format. Data is input serially, as described in the SISO section above. Once the data has been clocked in, it may be either read off at each output simultaneously, or it can be shifted out

In this configuration, each flip-flop is edge triggered. The initial flip-flop operates at the given clock frequency. Each subsequent flip-flop halves the frequency of its predecessor, which doubles its duty cycle. As a result, it takes twice as long for the rising/falling edge to trigger each subsequent flip-flop; this staggers the serial input in the time domain, leading to parallel output.

In cases where the parallel outputs should not change during the serial loading process, it is desirable to use a latched or buffered output. In a latched shift register (such as the 74595) the serial data is first loaded into an internal buffer register, then upon receipt of a load signal the state of the buffer register is copied into a set of output registers. In general, the practical application of the serial-in/parallel-out shift register is to convert data from serial format on a single wire to parallel format on multiple wires.



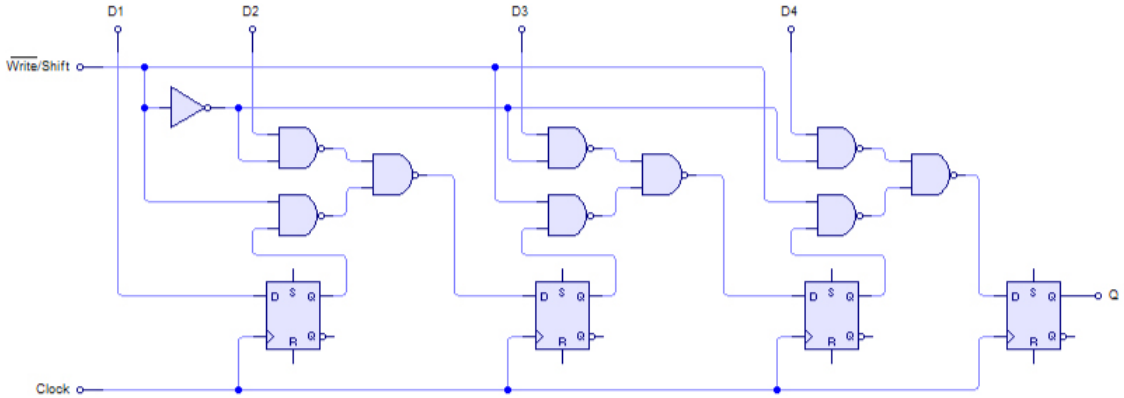
For N-bit of data,

Number of clock pulse required = N

Total time required = N.

Parallel In, Serial Out Shift Register

This configuration has the data input on lines D1 through D4 in parallel format, being D1 the MSB. To write the data to the register, the Write/Shift control line must be held LOW. To shift the data, the W/S control line is brought HIGH and the registers are clocked. The arrangement now acts as a SISO shift register, with D1 as the Data Input. However, as long as the number of clock cycles is not more than the length of the data-string, the Data Output, Q, will be the parallel data read off in order.



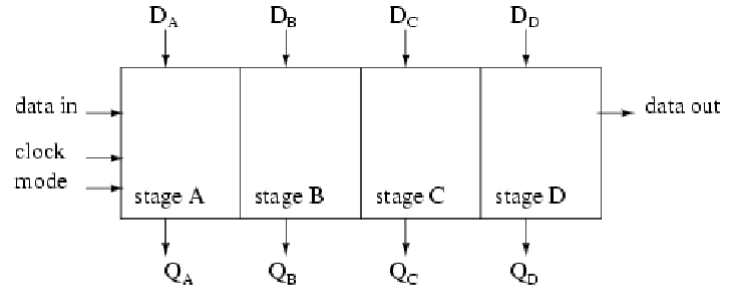
Parallel In, Parallel Out Shift Register

In this type of register, the data is entered into the register in parallel form and also the data is taken out of the register in parallel form. Immediately following the simultaneous entry of all data bits, the bits appear on the parallel outputs.

Data is applied to the D input terminals of the FFs. When a clock pulse is applied, at the positive going edge of that pulse, the D inputs are shifted out into the Q outputs of the FFs. The register now stores the data. The stored data is available instantaneously for shifting out in parallel form.

For N bit of data,

Number of clock pulse required = 0



Parallel-in, parallel-out shift register with 4-stages

PLATFORM USED:

1. Xilinx 9.1 ISE Design Suite
2. Spartan 3E XC3s250E 4C

FAQ:

Explain various types of delays in VHDL ?

The Various types of delays in VHDL are :-

Delta delay - In VHDL simulations, all signal assignments occur with some infinitesimal delay, known as delta delay. VHDL uses the concept of delta delay to keep track of processes that should occur at a given time step, but are actually evaluated in different machine cycles .A delta delay is a unit of time as far as the simulator hardware is concerned, but in the simulation itself time has no advance. Technically, delta delay is of no measurable unit, but from a hardware design perspective one should think of delta delay as being the smallest time unit one could measure, such as a femtosecond(fs).

Inertial delay - The inertial delay causes the pulses less than specified delay to get suppressed & will not propogate these pulses to change the output. The inertial delay model is specified by adding an after clause to the signal assignment statement. Inertial delay is basically a default delay, i.e it's a component delay.

Transport delay - Tranport delay adds the propogation delay to the signal. The transport delay model just delays the change in the output by the time specified in the after clause. Transport delay basically represents a wire delay. e.g. q <=transport a nor b after 1ns ;

What is the difference between Concurrent & Sequential Statements ?

Concurrent statements define interconnected processes and blocks that together describe a design's overall behavior or structure. They can be grouped using block statement. Groups of blocks can also be partitioned into other blocks. At the same level, a VHDL component can be connected to define signals within the blocks It is a reference to an entity A process can be a single signal assignment statement or a series of sequential statements (SS) Within a process, procedures and functions can partition the sequential statements

What is the difference between Signal and the Variable?

Signals are interpreted as wires or wires with memory (i.e., FFs, latches etc.) Signal are declared as :-

signal signal_name, signal_name, ... : data_type;

Signal assignment :-

signal_name <= projected_waveform;

The Concept of variables is found in traditional programming languages, in which a name represents a symbolic memory location where a value can be stored and modified. There is NO direct mapping between a variable and a hardware component. Variables can be declared and used only inside a process.

Variable declaration:

variable variable_name, ... : data_type;

Variable assignment:

variable_name := value_expression;

Variables contains no timing information (immediate assignment) i.e. no waveform is possible for variables. Both signals and variables can be assigned initial values. Although useful in simulations, synthesis cannot deal with them

CONCLUSION:

Experiment No. 3

Title: To write VHDL code for FIFO memory. Simulate with test bench, synthesis, implement on PLD

PROBLEM STATEMENT:
To write VHDL code for universal shift register with mode selection input for SISO, SIPO, PISO, & PIPO modes. Simulate with test bench, synthesis, implement on PLD.

OBJECTIVE:
To understand the features of HDL

THEORY:

Standard FIFO
FIFOs (First In, First Out) are essentially memory buffers used to temporarily store data until another process is ready to read it. As their name suggests the first byte written into a FIFO will be the first one to appear on the output. Typically FIFOs are used when you have two processes that operate and a different rate. A common example is a high speed communications channel that writes a burst of data into a FIFO and then a slower communications channel that read the data as need to send it at a slower rate.
The FIFO module below has two settings that can be configured to adjust the width and depth of the FIFO. The DATA_WIDTH variable adjusts the size of the DataIn and DataOut buses so that you can write different sizes of bytes if needed and the FIFO_DEPTH variable adjusts how big the internal memory of the FIFO is.
In order to write data into the FIFO first push the data onto the DataIn bus and then strobe the WriteEn input high for one clock cycle. This will write whatever is on DataIn into the FIFOs internal memory. If writing in bulk the WriteEn signal can be left high while changing the data on the DataIn bus each clock cycle. When the Full flag goes high, this means that the FIFO's memory is full and will not accept any more writes until data is read using the ReadEn input. If data is written while the Full flag is high it will be ignored.
For a standard FIFO when you write the first byte into the FIFO nothing happens on the DataOut bus until the ReadEn signal is pulsed high for at-least one clock cycle to. Once a byte has been written into the FIFO the Empty flag will go low. To read the next byte from the FIFO strobe the ReadEn signal high for one clock cycle and the next byte of data will be available to read on the next clock cycle. When the last byte of data is pushed onto the DataOut bus the Empty flag will go high.

PLATFORM USED:
1. Xilinx 9.1 ISE Design Suite
2. Spartan 3E XC3s250E 4C

FAQ:
What are VHDL Subtypes ?
VHDL subtypes are used to constrain defined types. Constraints take the form of range constraints or index constraints. However, a subtype may include the entire range of the base

type. Assignments made to objects that are out of the subtype range generate an error at run time. The syntax and an example of a subtype declaration is shown below :-
SUBTYPE First_ten IS INTEGER RANGE 0 to 9;

What is Synthesis?
Synthesis represents the transformation of an abstract description into a more detailed description. In general, the term "synthesis" is used for the automated transformation of RT leveldescriptions into gate level representations. This transformation is mainly influenced by the set of basic cells that is available in the target technology. While simple operations like comparisonsand either/or decisions are easily mapped to boolean functions, more complex constructs like mathematical operators are mapped to a tool specific macro cell library first. This means that a number of adder, multiplier, etc. architectures are known to the synthesis tool and these designs are treated as if they were designed by the user.

CONCLUSION:

Experiment No. 4

Title: To write VHDL code for LCD interface. Simulate with test bench, synthesis, implement on PLD.

PROBLEM STATEMENT:
To write VHDL code for LCD interface. Simulate with test bench, synthesis, implement on PLD.

OBJECTIVE:
1. To Study the interfaces.
2. To study LCD interface for CPLD/FPGA

THEORY:
This LCD controller is a VHDL component for use in CPLDs and FPGAs. VHDL component allows simple LCD integration into practically any programmable logic application. Figure 1 depicts the controller implemented to interface between an LCD module and a user custom logic.

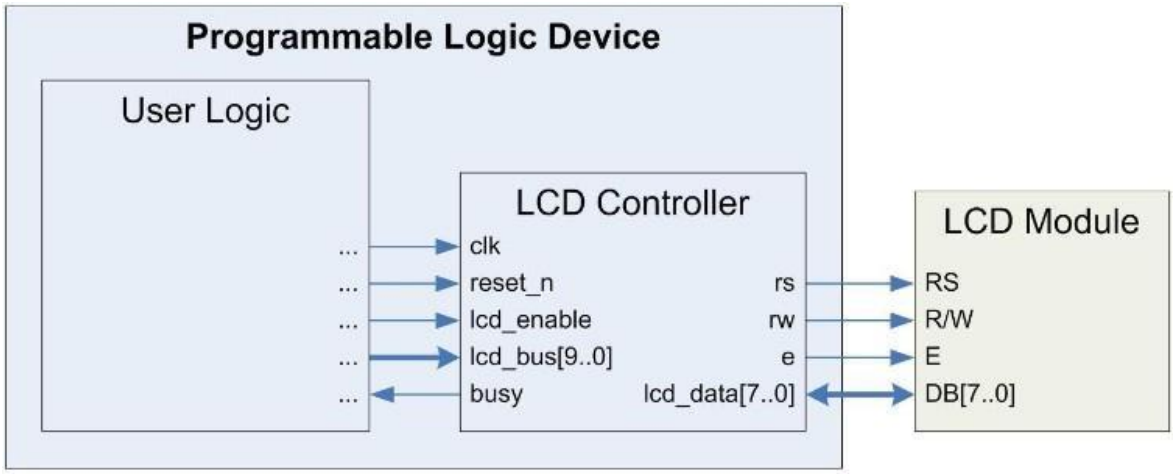


Figure 1. LCD Controller Implementation in PLD

State Machine:
The LCD controller state machine consists of five states. Upon start up, it immediately enters the Power-up state, where it waits 50ms to ensure the supply voltage has stabilized. It then proceeds to an Initialize state. The controller cycles the LCD through its initialization sequence, setting the LCD's parameters to default values defined in the hardware. This process completes in approximately 2.2ms, and the controller subsequently assumes a Ready state.
It waits in this state until the lcd_enable input is asserted, then advances to the Send state. Here, it communicates the appropriate information to the LCD, as defined by the lcd_bus input. After 50us, it returns to the Ready state until further notice. If a low logic level is applied to the reset_n input at any time for a minimum of one clock cycle, the controller resets to the Power-up state and re-initializes. Figure 2 illustrates the LCD controller state machine.

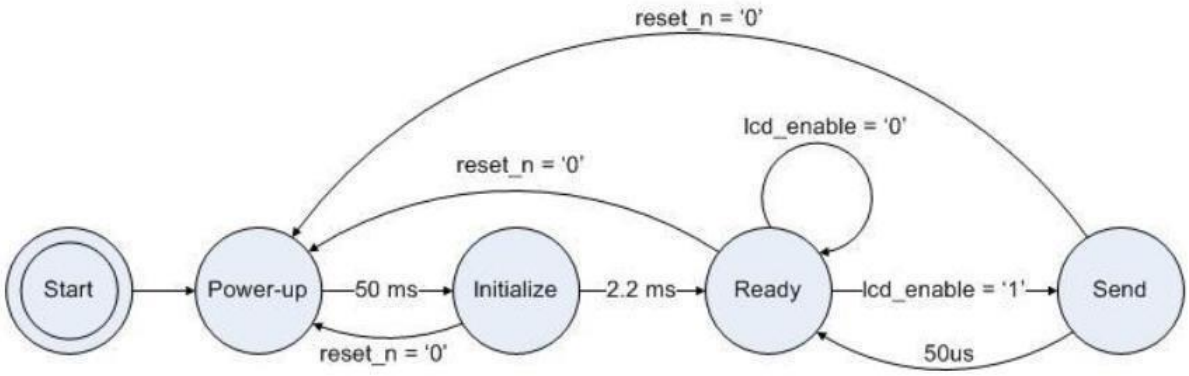


Figure 2. LCD Controller State Machine

completes in approximately 2.2ms, and the controller subsequently assumes a Ready state. It waits in this state until the lcd_enable input is asserted, then advances to the Send state. Here, it communicates the appropriate information to the LCD, as defined by the lcd_bus input. After 50us, it returns to the Ready state until further notice.If a low logic level is applied to the reset_n input at any time for a minimum of one clock cycle, the controller resets to the Power-up state and re-initializes.

I/O Name	Width	Mode	Description	Interface
clk	1	input	Clock for LCD controller. Default set for 50MHz. If a different frequency is desired, change the constant freq in the architecture declarations to reflect the new frequency in MHz.	system clock
reset_n	1	input	Active low synchronous reset pin. This pin must be set high to implement the LCD controller. Setting the pin low for one or more clock cycles restarts the LCD controller state machine.	user logic
lcd_enable	1	input	Data latch for LCD controller. H: initiates a transaction using the data currently on the lcd_bus, L: no transaction is initiated and any data on lcd_bus is ignored	user logic
lcd_bus	10	input	Data/instructions to be sent to the LCD module. The MSB is the rs signal, followed by the rw signal. The other 8 bits are the data bits. The LSB on the bus corresponds to the least significant data bit.	user logic
busy	1	output	Feedback on the state of the LCD controller. H: the controller is busy initializing or conducting a transaction with the LCD module, any instructions/data	user logic

			sent will be ignored, L: the controller is idle and ready to accept commands for a transaction	
rs	1	output	LCD module Register Select Signal; H: sending data, L: sending instructions	LCD pin 4
rw	1	output	LCD module Read/Write Select Signal; H: Read, L: Write	LCD pin 5
e	1	output	LCD module enable signal	LCD pin 6
lcd_data	8	bidir	Data bus to the LCD module / busy signal from the LCD	LCD pins 7-14

Initialization:

The LCD controller executes an initialization sequence each time it is powered-up or the reset_n pin is deasserted for a minimum of one clock cycle.

The controller asserts the busy pin during initialization. Once initialization completes, the busy pin deasserts, and the LCD controller waits in the Ready state for input from the user logic. The initialization sequence specifies several LCD parameters: function, display control, display clear, and entry mode. The LCD controller instantiates the following default set of these options.

- Function Set: 2 -line mode, display on
- Display Control: display on, cursor off, blink off
- Entry Mode: increment mode, entire shift off The user can send commands to the LCD to change any parameters after initialization. Alternatively, the user can edit the VHDL to change the default parameters. This simply requires commenting out the current VHDL line and uncommenting the line with the desired parameter setting. Table 2 lists the options available in the code.

Table 2. Initialization Options in the VHDL

Options	Choices	VHDL Line	Code
Function Set	2-line mode, display on*	93	lcd_data <= "00111100";
	1-line mode, display on	94	lcd_data <= "00110100";
	1-line mode, display off	95	lcd_data <= "00110000";
	2-line mode, display off	96	lcd_data <= "00111000";
Display ON/OFF	display on, cursor off, blink off*	104	lcd_data <= "00001100";
	display on, cursor off, blink on	105	lcd_data <= "00001101";
	display on, cursor on, blink off	106	lcd_data <= "00001110";
	display on, cursor on, blink on	107	lcd_data <= "00001111";
	display off, cursor off, blink off	108	lcd_data <= "00001000";
	display off, cursor off, blink on	109	lcd_data <= "00001001";
	display off, cursor on, blink off	110	lcd_data <= "00001010";

	display off, cursor on, blink on	111	lcd_data <= "00001011";
Entry Mode Set	increment mode, entire shift off*	127	lcd_data <= "00000110";
	increment mode, entire shift on	128	lcd_data <= "00000111";
	decrement mode, entire shift off	129	lcd_data <= "00000100";
	decrement mode, entire shift on	130	lcd_data <= "00000101";

PLATFORM USED:

1. Xilinx 9.1 ISE Design Suite
2. Spartan 3E XC3s250E 4C

FAQ:

What is VHDL? What are capabilities of VHDL?

VHDL is a programming language, much like C++, it has its own syntax and semantics. The big difference from traditional programming languages is that instead of describing instructions which a processor will execute, it describes how circuits should be organized. VHDL is basically a programming language used to model digital Systems. As it is emulating real hardware it is inherently parallel and also treats timing as important. This language is a commonly used in the design of field-programmable gate arrays. (FPGA) and application specific integrated circuits (ASIC). VHDL stands for Very High Speed Integrated Circuit Hardware Description language

What can be the various uses of VHDL ?

i) To synthesize digital circuits ii) To verify and validate digital designs
iii) To generate test vectors to test circuits iv) To simulate circuits

CONCLUSION:

Experiment No. 5

Title: To write VHDL code for up counter and down counter. Simulate with test bench, synthesis, implement on PLD.

PROBLEM STATEMENT:

To write VHDL code for UP and DOWN Counter, Simulate with test bench, synthesis, implement on PLD.

OBJECTIVE:

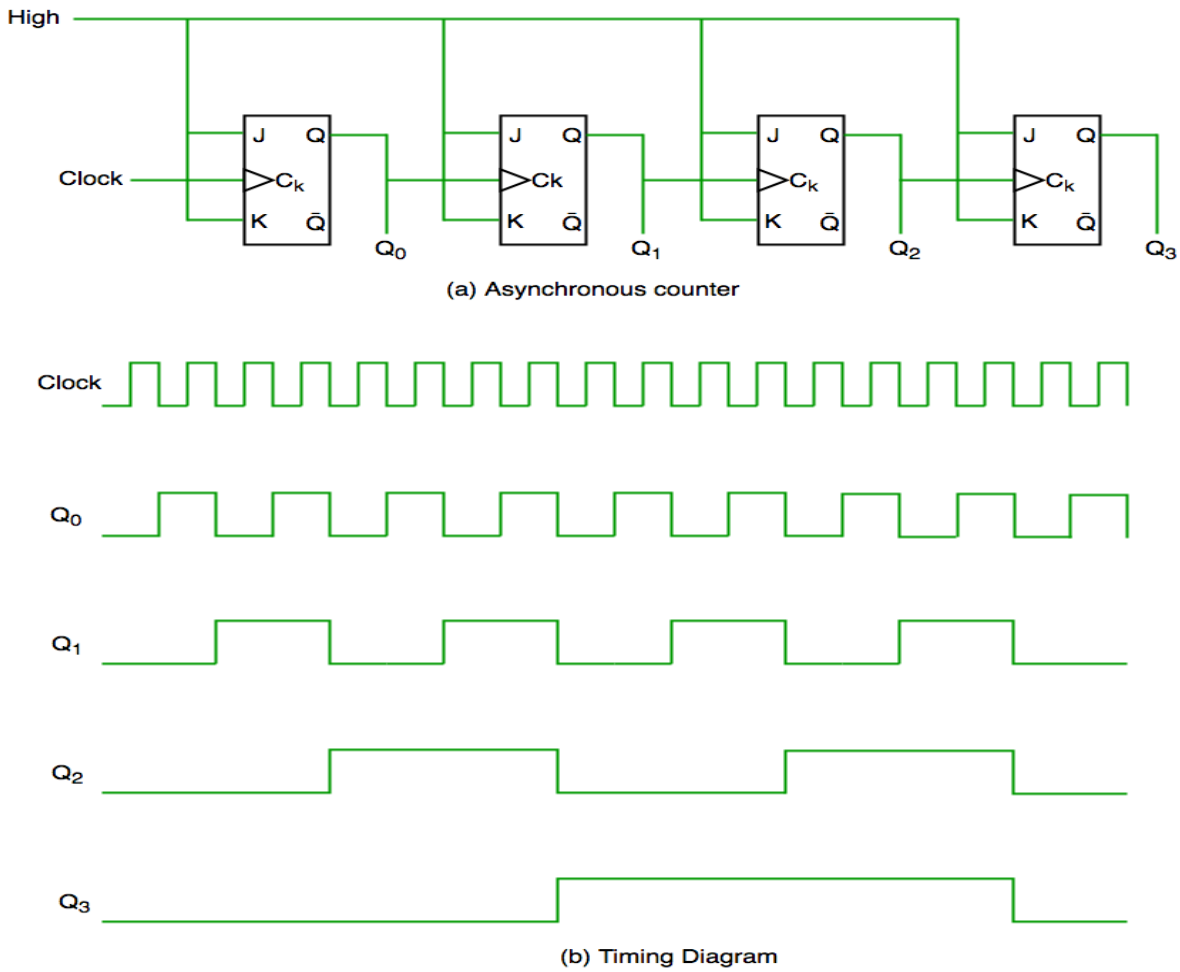
To Study the Up and Down Counter.
To study UP/ Down interface for CPLD/FPGA

THEORY:

A counter is a logic circuit that counts the incidences of particular events; it is identified traditionally by the frequencies of a clock pulse. Counters are employed in digital electronics in order to count the number of clocks which correspond to time frames, occurrences or particular operations. Counters are primitive blocks used in many systems and subsystems like digital clocks, timers, frequency counters, and memory address counters.

Counters can operate in different modes, primarily categorized into:
Up Counters: Increase the count value +1 for each of the clock pulse.
Down Counters: Subtract one from the count value with every clock pulse.
Up-Down Counters: In this, it can add or subtract with the help of an additional control signal which is unusual.
Counters are broadly divided into two categories

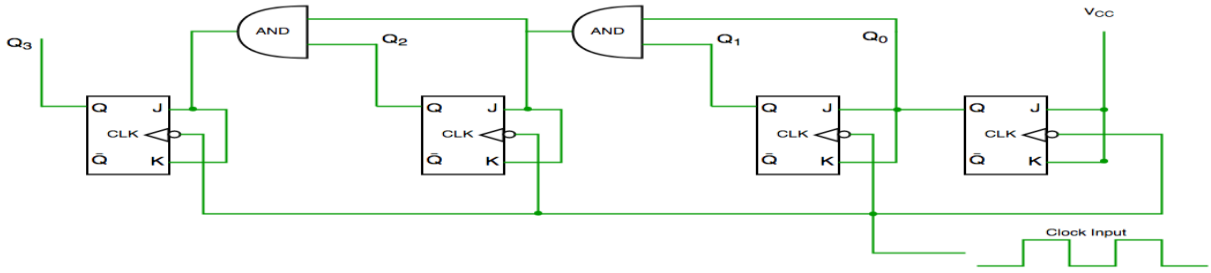
Asynchronous counter
Synchronous counter
1. Asynchronous Counter
In asynchronous counter we don't use universal clock, only first flip flop is driven by main clock and the clock input of rest of the following flip flop is driven by output of previous flip flops. We can understand it by following diagram-



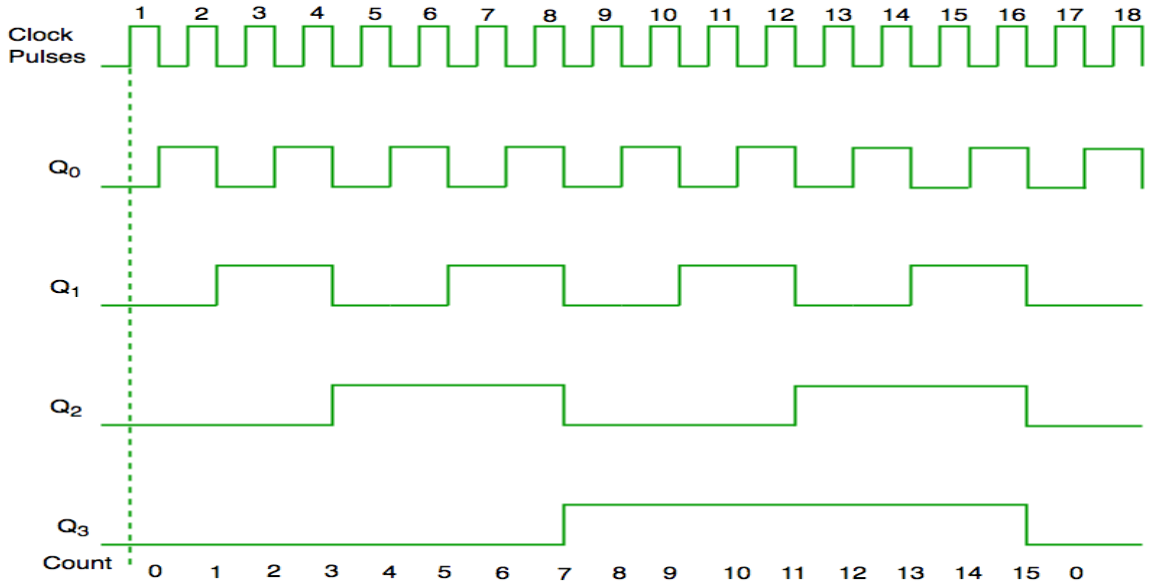
It is evident from timing diagram that Q0 is changing as soon as the rising edge of clock pulse is encountered, Q1 is changing when rising edge of Q0 is encountered(because Q0 is like clock pulse for second flip flop) and so on. In this way ripples are generated through Q0,Q1,Q2,Q3 hence it is also called RIPPLE counter and serial counter. A ripple counter is a cascaded arrangement of flip flops where the output of one flip flop drives the clock input of the following flip flop

2. Synchronous Counter

Unlike the asynchronous counter, synchronous counter has one global clock which drives each flip flop so output changes in parallel. The one advantage of synchronous counter over asynchronous counter is, it can operate on higher frequency than asynchronous counter as it does not have cumulative delay because of same clock is given to each flip flop. It is also called as parallel counter.



Synchronous counter circuit



PLATFORM USED:
Xilinx 9.1 ISE Design Suite
Spartan 3E XC3s250E 4C

FAQ:

1.What are the uses of up counters?
An up counter increases it's count with every clock pulse at it's input. Down counter decreases. A typical application for a up counter is a typical time of day clock.

2. Which flip-flop is used in counters?
In other words, if the output was 0 before the , it becomes 1 after the pulse; if it was 1, it becomes 0. By using T flip-flops in a counter, we can create a circuit that counts by toggling the output of each flip-flop on each clock pulse.

3. What is a synchronous up counter?

In synchronous counter, the clock input across all the flip-flops use the same source and create the same clock signal at the same time. So, a counter which is using the same clock signal from the same source at the same time is called Synchronous counter.

CONCLUSION:

Experiment No. 6

Title: To prepare CMOS layout for Inverter, NAND, NOR gates, Half Adder. Simulate with and without capacitive load, comment on rise,

PROBLEM STATEMENT:
To prepare CMOS layout for Inverter, NAND and NOR gate with and without capacitive load, comment on rise and fall time.

OBJECTIVE:

1. To manually design the mask layout of CMOS inverter, NAND, NOR gates, half adder
2. To check the design for design rule errors.
3. To check the functionality using simulation with the built-in simulator.
4. Measure propagation delay

THEORY:

MICROWIND program allows to design and simulate an integrated circuit. The package itself contains a library of common logic and analog ICs to view and simulate. MICROWIND includes all the commands for a mask editor as well as new original tools never gathered before in a single module. You can gain access to Circuit Simulation by pressing one single key. The electric extraction of your circuit is automatically performed and the analog simulator produces voltage and current curves immediately.

A specific command displays the characteristics of pMOS and nMOS, where the size of the device and the process parameters can be very easily changed. Altering the MOS model parameters and, then, seeing the effects on the Vds and Ids curves constitutes a good interactive tutorial on devices.

The MOS device

The MOS symbols are reported below. The n-channel MOS is built using polysilicon as the gate material and N+ diffusion to build the source and drain. The p-channel MOS is built using polysilicon as the gate material and P+ diffusion to build the source and drain.

CMOS Inverter:

CMOS configuration is called complementary MOS. Thecircuit topology is complementary push-pull in the sense that for high input, the NMOS transistor drives (pull-down), the output node, while the PMOS transistor act as the load, and for the low input the PMOS transistor drives (pull-up), output node, while NMOS acts as load. Consequently both devices contribute equally to the circuit operation characteristics.

The CMOS inverter has 2 input advantages over the other inverter configuration. The trend of increasing sub threshold leakage current in deep sub-micron technologies causes great design challenges. in all other inverter structures examined so far ,a non-zero steady state currentis drown from the power source when the driver transistor is turned-on, which result in a significant dc power consumption. The other advantages of CMOS configuration are that the voltage swing between 0v and VDD and that the Vth is usually very sharp.

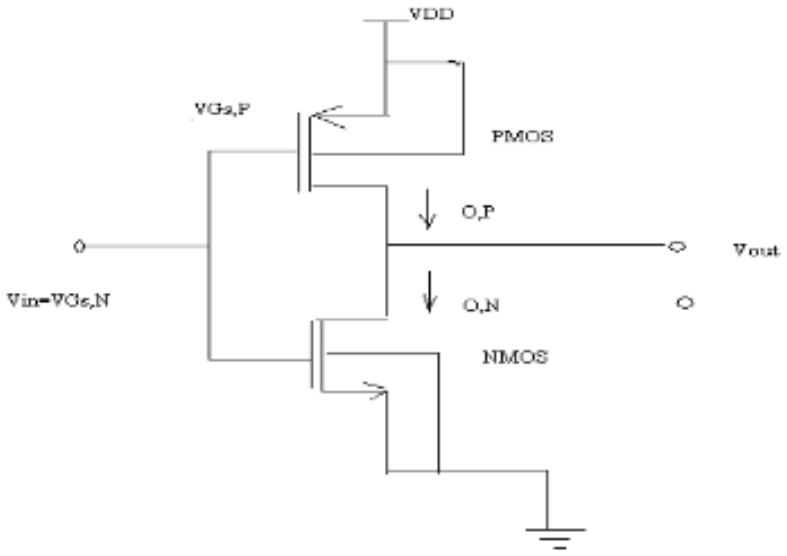


FIG. 1. CMOS INVERTER CIRCUIT

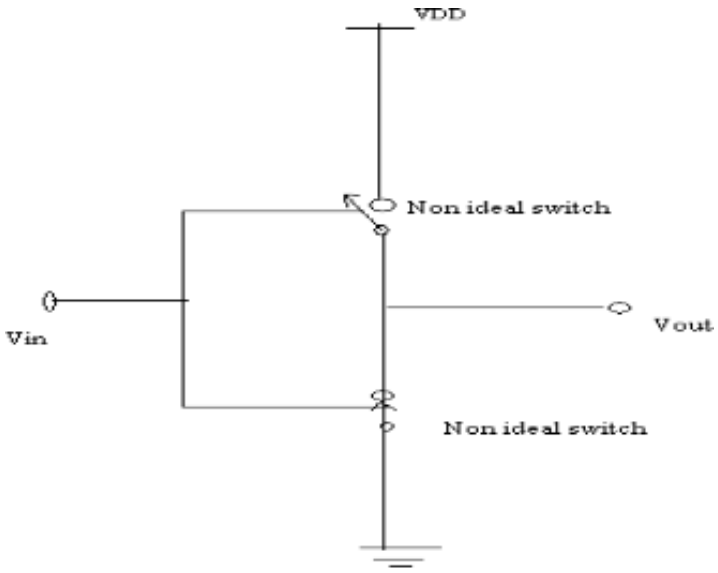


FIG 2.SIMPLIFIED VIEW OF CMOS

NAND and NOR Theory:

A two-input NAND gate (F = A·B). The PDN network consists of two NMOS devices in series that conduct when both A and B are high. The PUN is the dual network, and consists of two parallel PMOS transistors. This means that F is 1 if A = 0 or B = 0, which is equivalent to F = A·B. The truth table for the simple two input NAND gate is given in Table 1. It can be verified that the output F is always connected to either VDD or GND, but never to both at the same time.

Table no 1: Operating regions of CMOS Inverter

REGION	Vin	Vout	NMOS	PMOS
A	<VTO,n	VoH	Cut-off	linear
B	ViL	High~VoH	saturation	linear
C	VtH	VtH	saturation	saturation
D	ViH	Low~VoL	linear	saturation
E	>(VDD+VTo,p)	VoL	linear	Cut-off

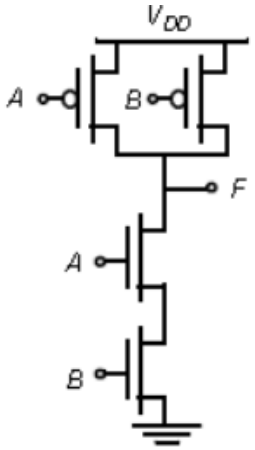


Fig 1: Two-input NAND gate in complementary static CMOS style.

A	B	F
0	0	1
0	1	1
1	0	1
1	1	0

Table 1: Truth Table for 2 input NAND

The first step in the synthesis of the logic gate is to derive the pull-down network. The fact is that NMOS devices in series implements the AND function and parallel device implements the OR function. The next step is to use duality to derive the PUN in a hierarchical fashion. The PDN network is broken into smaller networks (i.e., subset of the PDN) called sub-nets that simplify the derivation of the PUN.

Design Consideration: The important point to take away from is that the noise margins are input pattern dependent. A smaller input glitch will cause a transition at the output if only one of the inputs makes a transition. Therefore, this condition has a lower low noise margin. A common practice when characterizing gates such as NAND and NOR is to connect all the inputs together. This unfortunately does not represent the worst-case static behavior. The data dependencies should be carefully modeled.

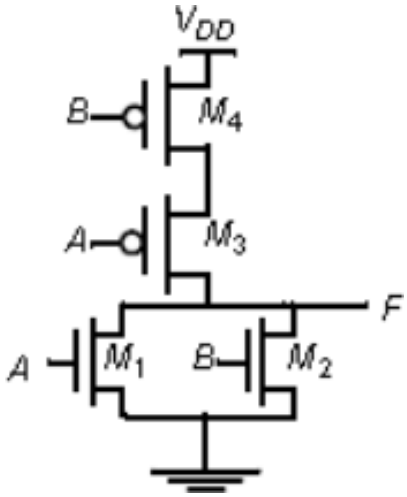


Fig2: Two-input NOR gate in complementary static CMOS style.

X	Y	X NOR Y
0	0	1
0	1	0
1	0	0
1	1	0

Table 2: Truth Table for 2 input NOR

The output of this NOR network is high, if and only if both inputs A and B are low. The worst-case pull-down transition happens when only one of the NMOS devices turns on (i.e., if either A or B is high). Since the pull-down path in the worst case is a single device, the NMOS devices (M1 and M2) can have the same device widths as the NMOS device in the inverter. For the output to be pulled high, both devices must be turned on. Since the resistances add, the devices must be made two times larger compared to the PMOS in the inverter. Since PMOS devices have a lower mobility relative to NMOS devices, stacking devices in series must be avoided as much as possible. A NAND implementation is clearly preferred over a NOR implementation for implementing generic logic

PLATFORM USED:

1. Microwind 3.1

FAQ:

Why is NAND gate preferred over NOR gate for fabrication?

NAND is a better gate for design than NOR because at the transistor level the mobility of electrons is normally three times that of holes compared to NOR and thus the NAND is a faster gate. Additionally, the gate-leakage in NAND structures is much lower.

What happens when the PMOS and NMOS are interchanged with one another in an inverter?

If the source & drain also connected properly...it acts as a buffer. But suppose input is logic 1 O/P will be degraded 1 Similarly degraded 0;

CONCLUSION:

Experiment No. 7

Title: To prepare CMOS layout for 2:1 multiplexer using logic gates and transmission gates. Simulate with and without capacitive load, comment on rise, and fall times.and fall times.

PROBLEM STATEMENT:

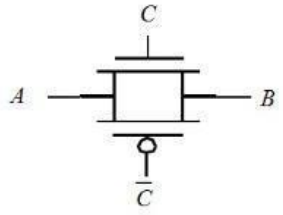
To prepare CMOS layout for 2:1 MUX using logic gate and transmission gate and compare the performance in terms of speed, space and power

OBJECTIVE:

1. To Study Logic gate implementation using conventional approach.
2. To Study Logic gate implementation using transmission gate approach.
3. To compare the performance among the design.

THEORY:

A transmission gate has three inputs, called *source*, *n-gate*, and *p-gate*; and it has one output, called *drain*. When diagrammed, the *source* input and *drain* output are drawn connected by two plates. The two *gate* inputs are drawn as lines connected to plates parallel to each of the plates connecting *source* to *drain*. The *p-gate* input's line has a circle, while the *n-gate* input's line does not.



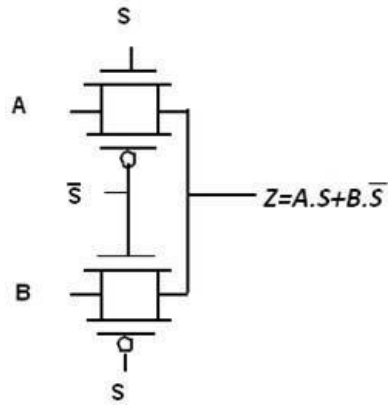
The transmission gate is simply the combination of two complementary transistors. The values at *n-gate* and *p-gate* are expected to be opposite to each other. If *p-gate* is 0 while *n-gate* is 1, then the value found at *source* is transmitted to *drain*. If *p-gate* is 1 while *p-gate* is 0, then the connection is broken, so the value at *drain* is left floating. In all other cases, *drain* receives an error output — unless *source* is floating, in which case *drain* is floating as well. This behavior is summarized by the following table.

p-gate	***n-gate***	***drain***
0	0	X*
0	1	*source*
1	0	Z
1	1	X*
X/Z	*any*	X*
any	X/Z	X*

** If source is Z, drain is Z; otherwise drain is X.*

If the Data Bits attribute is more than 1, each gate input is still a single bit, but the gate values are applied simultaneously to each of the source input's bits.

The transmission gate acts as a bidirectional switch controlled by the gate signal *C*. When *C=1*, both MOSFETs are on, allowing the signal to pass through the gate. In short, *A=B, if C=1*. On the other hand, *C=0*, places both transistors in cut-off, creating an open circuit between nodes *A* and *B*. Fig.5 shows the implementation of a 2:1 MUX using transmission gate logic.



Here, the transmission gates selects input *A* or *B* on the basis of the value of the control signal *S*. When *S=0, Z=A* and when *S=1, Z=B*.

PLATFORM USED:

1. Microwind 3.1

FAQ:

Why PMOS and NMOS are sized equally in a Transmission Gates?

In Transmission Gate, PMOS and NMOS aid each other rather competing with each other. That's the reason why we need not size them like in CMOS. In CMOS design we have NMOS and PMOS competing which is the reason we try to size them proportional to their mobility.

Why don't we use just one NMOS or PMOS transistor as a transmission gate?

NMOS passes a good 0 and a degraded 1, whereas PMOS passes a good 1 and bad 0. for pass transistor, both voltage levels need to be passed and hence both NMOS and PMOS need to be used.

CONCLUSION:

Experiment No. 8

Title: To prepare CMOS layout for Single bit SRAM cell. Simulate with and without capacitive load, comment on rise, and fall times.

PROBLEM STATEMENT:
Design a layout for Single Bit SRAM Cell using CMOS Technology.

OBJECTIVE:
1. To Study the SRAM.
2. To study and layout design for single bit SRAM cell.
3. To study capacitive effect.

THEORY:

Memory design
Memories are usually constructed as two dimensional arrays of bits. Thus a memory containing 2w words each of 2b bits will be configured as 2w rows by 2b columns. W address bits will be decoded to give the row and either the whole word will be output or multiplexor used to select a single bit using a further b address bits.

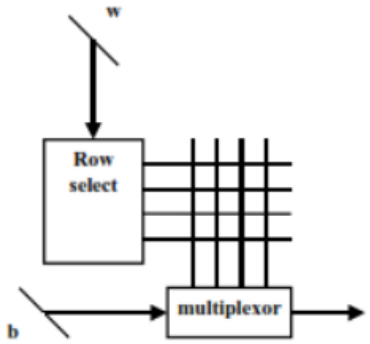


Fig 1:Basic memory storage operation

Read-only memory (ROM)
A read-only memory (ROM) is like a PLA with all the possible minterms being calculated. The individual memory cells can be very compact.

Programmable read-only memories (PROMs)
Programmable read-only memories (PROMs) allow the diffusion tabs to be switched in electrically. Erasable PROMs allow this switching to be reversed, either by exposure to ultraviolet light (EPROMs) or under digital control (electrically erasable PROMs or EEPROMs). **Static read/write memory**
The simplest form of writeable memory (RAM) is static memory. A bit is stored in a pair of Cross-coupled invertors, with separate circuits to control the reading and writing of the data. The

memory has two independent ports for reading; both selection lines are opened for writing. Six transistors are required to store each bit, plus some overheard for the control circuitry.

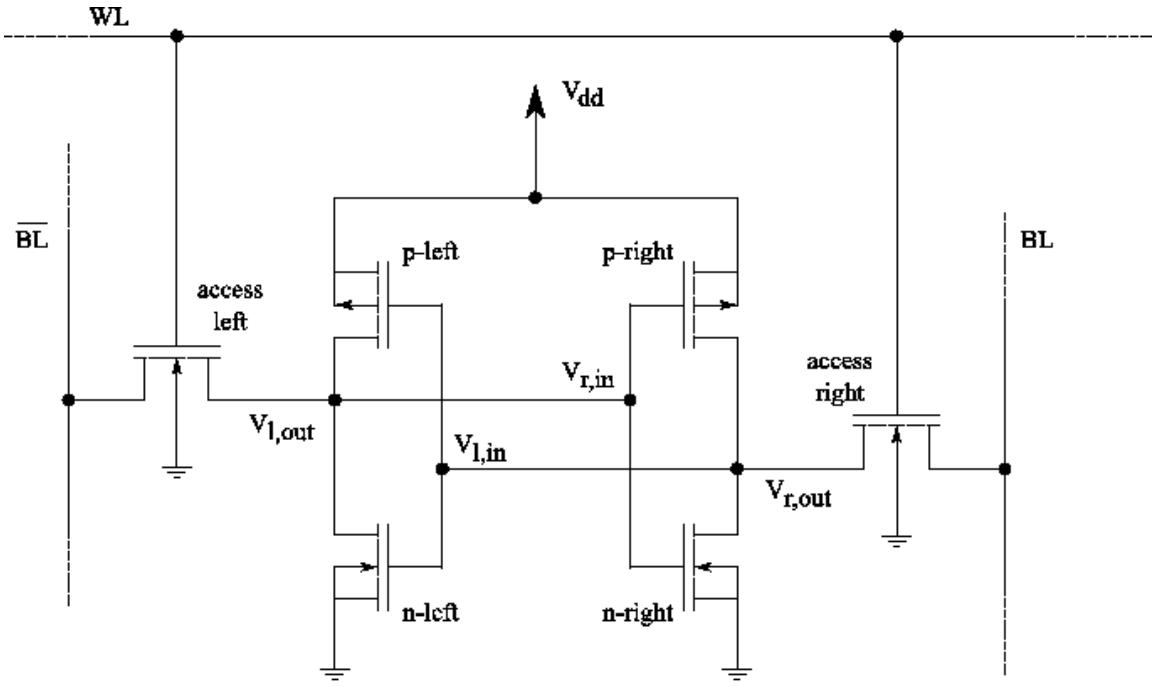
6T SRAM Cell

Static random access memory (SRAM) can retain its stored information as long as power is supplied. This is in contrast to dynamic RAM (DRAM) where periodic refreshes are necessary or non-volatile memory where no power needs to be supplied for data retention, as for example flash memory. The term ``random access" means that in an array of SRAM cells each cell can be read or written in any order, no matter which cell was last accessed. The structure of a 6 transistor SRAM cell, storing one bit of information, can be seen in Figure. The core of the cellis formed by two CMOS inverters, where the output potential of each inverter Vout is fed as input into the other Vin. This feedback loop stabilizes the inverters to their respective state.

The access transistors and the word and bit lines, WL and BL, are used to read and write from or to the cell. In standby mode the word line is low, turning the access transistors off. In this state the inverters are in complementary state. When the p-channel MOSFET of the left inverter is turned on, the potential V1,out is high and the p-channel MOSFET of inverter two is turned off Vr,out, is low.

To write information the data is imposed on the bit line and the inverse data on the inverse bit line BL*bar*. Then the access transistors are turned on by setting the word line to high. As the driver of the bit lines is much stronger it can assert the inverter transistors. As soon as the information is stored in the inverters, the access transistors can be turned off and the information in the inverter is preserved.

For reading the word line is turned on to activate the access transistors while the information is sensed at the bit lines.



Circuit of a 6 transistor SRAM cell. It consists of two CMOS inverters and two access MOSFETs. NBT stress mainly affects the p-channel transistors.

PLATFORM USED:

1. Microwind 3.1

FAQ:

What is slack?

The slack is the time delay difference from the expected delay(1/clock) to the actual delay in a particular path.

What is Body Effect?

In general multiple MOS devices are made on a common substrate. As a result, the substrate voltage of all devices is normally equal. However while connecting the devices serially this may result in an increase in source-to-substrate voltage as we proceed vertically along the series chain (Vsb1=0, Vsb2 0).Which results Vth2>Vth1.

CONCLUSION:

Experiment No. 9

Title: To prepare CMOS layout for Half and Full Adder. Simulate with andwithout capacitive load, comment on rise, and fall times.

PROBLEM STATEMENT:
Design a layout for Half and Full adder using CMOS Technology.

OBJECTIVE:

1. To Study the Half and Full Adder
2. Layout design of a Half and Full adder

THEORY:

HALF Adder

The half adder is a basic building block for more complex adder circuits such as full adders and multiple-bit adders. It performs binary addition of two single-bit inputs, A and B, and provides two outputs, SUM and CARRY.

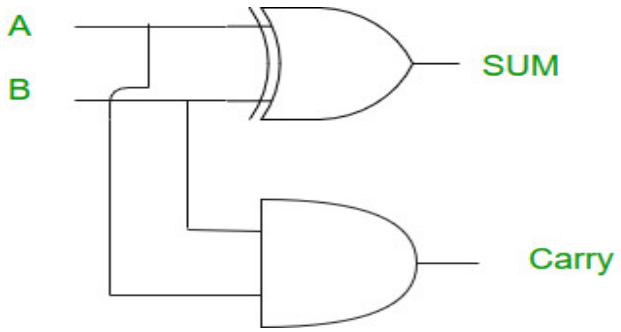
The SUM output is the least significant bit (LSB) of the result, which is the XOR of the two inputs A and B. The XOR gate implements the addition operation for binary digits, where a "1" is generated in the SUM output only when one of the inputs is "1".

The CARRY output is the most significant bit (MSB) of the result, indicating whether there was a carry-over from the addition of the two inputs. The CARRY output is the AND of the two inputs A and B. The AND gate generates a "1" in the CARRY output only when both inputs are "1".

Half adder is the simplest of all adder circuits. Half adder is a combinational arithmetic circuit that adds two numbers and produces a sum bit (s) and carry bit (c) both as output. The addition of 2 bits is done using a combination circuit called a Half adder. The input variables are augend and addend bits and output variables are sum & carry bits. A and B are the two input bits.

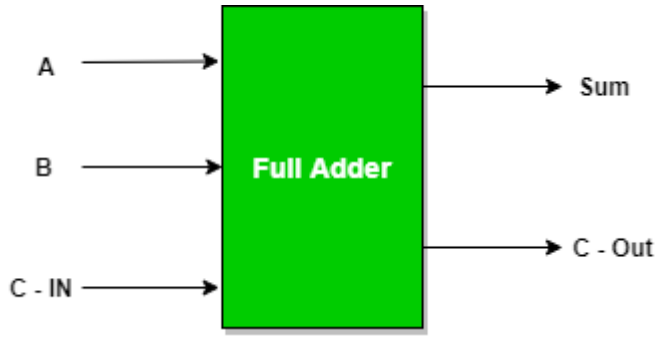
Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Full Adder

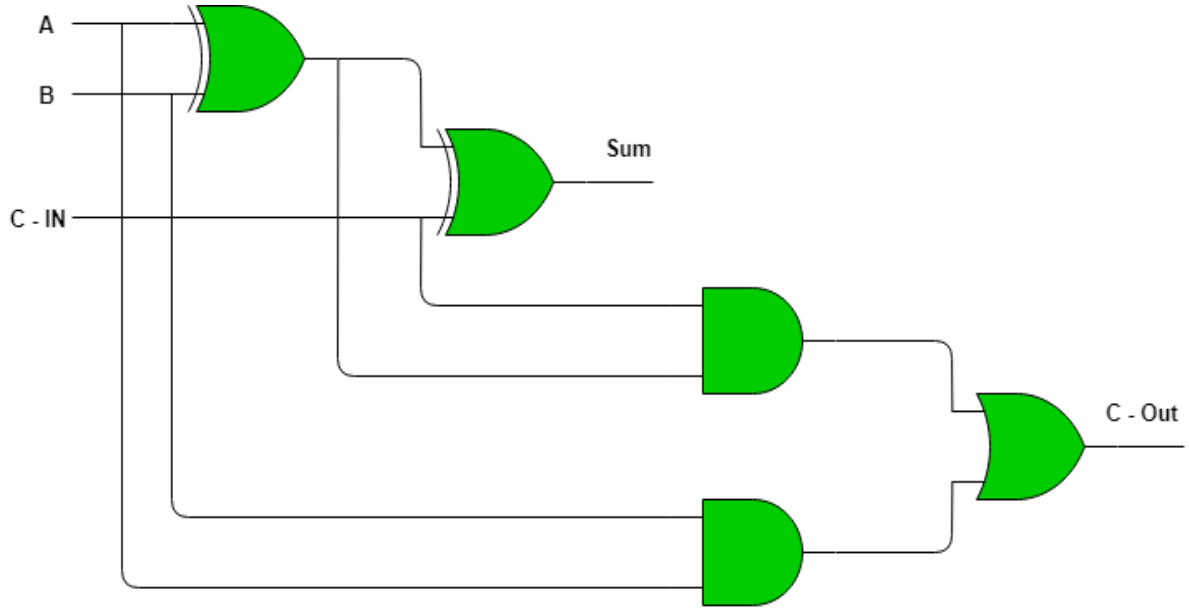
Full Adder is the adder that adds three inputs and produces two outputs. The first two inputs are A and B and the third input is an input carry as C-IN. The output carry is designated as C-OUT and the normal output is designated as S which is SUM. The C-OUT is also known as the majority 1's detector, whose output goes high when more than one input is high. A full adder logic is designed in such a manner that can take eight inputs together to create a byte-wide adder and cascade the carry bit from one adder to another. we use a full adder because when a carry-in bit is available, another 1-bit adder must be used since a 1-bit half-adder does not take a carry-in bit. A 1-bit full adder adds three operands and generates 2-bit results.



Full Adder Truth Table:

Inputs			Outputs	
A	B	C – IN	Sum	C - Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Logic Diagram:



FAQ:

Why are PMOS transistor networks generally used to produce high signals, while NMOS networks are used to product low signals?

This is because threshold voltage effect. A NMOS device cannot drive a full 1 or high and PMOS can't drive full '0' or low. The maximum voltage level in NMOS and minimum voltage level in PMOS are limited by threshold voltage. Both NMOS and PMOS do not give rail to rail swing.

What are set up time & hold time constraints? What do they signify?

Setup time: Time before the active clock edge of the flip flop, the input should be stable. If the signal changes state during this interval, the output of that flip flop can't be predictable (called metastable).

Hold Time: The after the active clock edge of the flip flop, the input should be stable. If the signal changes during this interval, the output of that flip flop can't be predictable (called metastable).

CONCLUSION: